

Cloud Service Benchmarking (Winter Term 2025/26)

Prof. Dr.-Ing. D. Bermbach, N. Japke, T. C. Rese, D. Baumann

Benchmarking Assignment

In the benchmarking assignment, you will design and implement an application benchmark that runs fully in the cloud and assists in answering a research question about an open-source system under test (SUT) of your choice. You are free to design the benchmark according to your wishes as long as you follow the general requirements defined below.

While you work on this assignment, there are several deliverables that you need to provide us to assess the completion of your benchmark. These include presentations, code + analysis scripts, measurement data, and a final report. Deadlines for all deliverables can be found below.

Requirements

1. You must use an open-source SUT, which you set up and configure yourself. Managed or pre-hosted software solutions (such as *DynamoDB* or pre-hosted *MongoDB*, *Kubernetes*, etc.) are not allowed.
2. You need to implement a benchmarking tool from scratch. This includes a load generator for your benchmark, which stresses the SUT (usually with multiple workers in parallel). You need to ensure that the SUT is sufficiently stressed in your experiments. You can use libraries such as *Faker* to generate data and messaging libraries to send requests to your SUT. It is not allowed to use ready-to-go load generators such as *k6* or *locust*.
3. The SUT and load generator must be deployed on different cloud instances. You can use your own notebook to manage the benchmark orchestration and analyze the benchmark result files, however, both SUT and load generator must be deployed in the cloud and on different machines.
4. You need to collect metrics concerning runtime behavior of the SUT during a benchmarking experiment. These metrics depend on your research question and form the basis for further analysis. Most important here is to show that your SUT is fully “utilized” (according to the metric that you are looking at) during the benchmark run.
5. You must ensure the repeatability of your benchmarks by executing and analyzing at least three (repeated) experiment runs.
6. Results from your benchmarking experiments need to be refined and presented using appropriate figures, plots, and tables. Don’t forget to report error bars, confidence intervals, or other measures of uncertainty for your experiment results.

Deliverables

1. Presentations (slides need to be handed in using ISIS).
 - a. Two milestone presentations, which are about your current progress in completing the benchmarking assignment. The presentations last approx. 2min, with strictly one PDF slide.
 - i. In the first milestone presentation you need to present your chosen SUT and research question. Due **November 18, 2025 (Tue)**.
 - ii. In the second milestone presentation you need to present the design of your benchmark. The implementation needs to be completed by this time, but you do not need to be finished with your analysis yet. Due **January 6, 2026 (Tue)**.
 - b. Final presentation, where you present your whole work in detail (SUT + research question, benchmark design, evaluation, discussion). The duration is 10min presentation followed by 5min time for Q&A. Due **February 4, 2026 (Wed)**.
2. Code artifacts (benchmark + analysis scripts). You also need to provide sufficient documentation for us to run all your code! Due **February 4, 2026 (Wed)**.
3. Measurements (raw data) and results (preliminary, figures and transformed data). Due **February 4, 2026 (Wed)**.
4. Final report (ca. 6-8 pages, IEEE conference template, 10pt). Make sure to include all sections expected in a scientific report (incl. abstract), make use of figures and tables, and use proper citations for references! There will be some example reports on the ISIS course page. Due **March 8, 2026 (Sun)**.

Further Notes

- You can build your application using most standard programming languages and libraries, except tooling designed to generate benchmarking loads for you. Part of this task is the implementation of benchmark components, so using off-the-shelf solutions is not allowed. Make sure to pick a technology stack that properly supports multi-threading.
- Don't plagiarize. If you reuse source code from somewhere else provide a reference (like you would with a citation in a written text). Likewise for your report. Don't use ChatGPT.
- If you want to use Google Cloud, you can get a 50\$ grant code by filling out the form on the ISIS course page **until November 9, 2025 (Sun)**.